MARKET77

Fase 1: base SaaS, catálogo e estoque

Plano de implementação

Next.js, TypeScript e Postgres com Row Level Security

10 tarefas, TDD e commits frequentes

Data: 01/10/2026

Spec: docs/superpowers/specs/2026-10-01-market77-design.md

Resumo das tarefas

#	Tarefa	Passos
1	Scaffold, banco de testes e runner de migrations	9
2	Schema, RLS e `withOrg`	7
3	Utilitários, senhas e sessões	7
4	Permissões	5
5	Serviço de autenticação e guard	6
6	Domínio de mercados e produtos	7
7	Preço por mercado e estoque	7
8	Interface web: entrar, cadastrar e navegação da organização	6
9	Interface web: mercados, produtos, estoque e equipe	5
10	Verificação final e README	4

Como usar este plano. Cada tarefa segue o ciclo: escrever o teste, vê-lo falhar, implementar o mínimo, vê-lo passar e fazer commit. Os blocos de código são exatamente o que deve ser criado. O plano completo, com os mesmos passos, está em Markdown em `docs/superpowers/plans/`.

> **For agentic workers:** REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (`- [ ]`) syntax for tracking.

Objetivo	Entregar a base multi-tenant do Market77: contas com vários mercados, papéis, catálogo de produtos com códigos de barras, preço por mercado e estoque por livro-razão, com painel web mínimo.
Arquitetura	Next.js (App Router) com Postgres. O isolamento entre clientes é feito por Row Level Security: toda requisição de negócio roda em uma transação que define `app.org_id`, e o papel de banco da aplicação não ignora RLS. Autenticação própria (e-mail e senha com `scrypt`, sessões opacas guardadas com hash no banco). A lógica de domínio fica em funções que recebem a transação, e a autorização fica na camada de actions.
Stack	Next.js 15, React 19, TypeScript, Postgres 16 (Docker), `postgres` (postgres.js), `zod`, `vitest`, `tsx`. Migrations em SQL puro, sem ORM.
Spec	`docs/superpowers/specs/2026-10-01-market77-design.md` (seções 3, 4 e 8, fase 1)

Restrições globais

- Stack: Next.js + TypeScript, Postgres com Row Level Security (spec seção 2).
- Toda tabela de negócio tem `org_id` e RLS; o isolamento é por `org_id` (spec seção 4).
- Estoque só muda por `stock_movements`; o saldo é a soma dos movimentos e nunca é editado diretamente (spec seção 4).
- Preço e valores monetários são inteiros em centavos (`price_cents`).
- Papéis: dono (`owner`), gerente (`manager`), repositor (`restocker`) (spec seção 3).
- Cliente final anônimo: não existe login de cliente; só usuários do painel (spec seção 2).
- Interface do painel em português do Brasil.
- Fora desta fase: terminais, agente, vendas, pagamentos, lotes e validade, NFC-e, controle de porta (spec seções 8 e 9).
- Nenhuma credencial real no repositório; apenas senhas de desenvolvimento local.

Foco de revisão

1. Referência entre tenants: o tenant A tenta criar `market_products` ou movimento apontando para UUID do tenant B (FK composta deve barrar). Esperado: erro `not_found`, nenhum dado criado. Teste na Tarefa 2 e na Tarefa 7.
2. Conexão reutilizada do pool com `app.org_id` vazio depois de uma transação: consulta sem contexto deve retornar zero linhas, nunca erro de cast para uuid nem dados de outro tenant. Teste na Tarefa 2.
3. E-mail duplicado com caixa ou espaços diferentes (`ANA@x.com` versus `ana@x.com`): deve contar como o mesmo e-mail. Teste na Tarefa 5.
4. Mesmo código de barras em dois tenants é permitido; no mesmo tenant é rejeitado e a criação do produto é desfeita por completo. Teste na Tarefa 6.
5. Movimentos de estoque simultâneos: 20 baixas concorrentes devem resultar no saldo exato. Teste na Tarefa 7.

6. Preço zero, negativo ou fracionado e `orgId` que não é UUID na URL: devem ser rejeitados sem erro 500. Testes nas Tarefas 5 e 7.

Estrutura de arquivos

```
package.json, tsconfig.json, vitest.config.ts, docker-compose.yml, .env.example, .gitignore, README.md
docker/init.sql                     # cria o banco de testes
scripts/migrate.ts                  # roda migrations no banco de desenvolvimento
src/db/client.ts                    # pools: adminSql (dono) e appSql (sujeito a RLS)
src/db/migrate.ts                   # runner de migrations SQL
src/db/migrations/0001_init.sql     # schema, RLS e grants
src/db/tenant.ts                    # withOrg(orgId, fn)
src/lib/errors.ts                   # AppError e detecção de erros do Postgres
src/lib/validate.ts                 # parse (zod) e formToObject
src/lib/money.ts                    # parseBRL e formatBRL
src/lib/messages.ts                 # códigos de erro em português
src/auth/password.ts                # hash e verificação de senha (scrypt)
src/auth/sessions.ts                # criar, ler e apagar sessão
src/auth/permissions.ts             # papéis e can()
src/auth/service.ts                 # signUp, login, addMember, memberships
src/auth/guard.ts                   # requireUser, requireOrg (usa cookies do Next)
src/domain/markets.ts               # createMarket, listMarkets
src/domain/products.ts              # createProduct, listProducts
src/domain/market-products.ts       # setMarketProduct, listMarketProducts
src/domain/stock.ts                 # recordMovement, getBalance, listLowStock
src/app/...                         # páginas e server actions (Tarefas 8 e 9)
tests/...                           # espelha src/
```

TAREFA 1

Scaffold, banco de testes e runner de migrations

Arquivos:

- Criar: `package.json`, `tsconfig.json`, `vitest.config.ts`, `docker-compose.yml`, `docker/init.sql`, `.env.example`, `.gitignore`
- Criar: `src/db/client.ts`, `src/db/migrate.ts`, `src/db/migrations/.gitkeep`, `scripts/migrate.ts`
- Criar: `tests/global-setup.ts`, `tests/helpers.ts`, `tests/db.test.ts`

Interfaces:

- Produz: `adminSql`, `appSql` (de `@/db/client`), `migrate(sql: Sql): Promise<string[]>` (de `@/db/migrate`), `resetDb()` e `makeOrg(name?: string): Promise<string>` (de `tests/helpers`).

[ ] Passo 1: Criar `package.json`

```
{
  "name": "market77",
  "private": true,
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start",
    "test": "vitest run",
    "typecheck": "tsc --noEmit",
    "db:migrate": "tsx --env-file=.env scripts/migrate.ts"
  },
  "dependencies": {
    "next": "^15.0.0",
    "postgres": "^3.4.0",
    "react": "^19.0.0",
    "react-dom": "^19.0.0",
    "zod": "^3.23.0"
  },
  "devDependencies": {
    "@types/node": "^22.0.0",
    "@types/react": "^19.0.0",
    "@types/react-dom": "^19.0.0",
    "tsx": "^4.19.0",
    "typescript": "^5.6.0",
    "vitest": "^2.1.0"
  }
}
```

[ ] Passo 2: Criar `tsconfig.json`, `.gitignore`, `.env.example`

`tsconfig.json`:

```
{
  "compilerOptions": {
    "target": "ES2022",
    "lib": ["dom", "dom.iterable", "esnext"],
    "skipLibCheck": true,
    "strict": true,
    "noEmit": true,
    "esModuleInterop": true,
    "module": "esnext",
    "moduleResolution": "bundler",
    "resolveJsonModule": true,
    "isolatedModules": true,
    "jsx": "preserve",
    "incremental": true,
    "plugins": [{ "name": "next" }],
    "paths": { "@/*": ["./src/*"] }
  },
  "include": ["next-env.d.ts", "**/*.ts", "**/*.tsx", ".next/types/**/*.ts"],
  "exclude": ["node_modules"]
}
```

`.gitignore`:

```
node_modules
.next
.env
*.tsbuildinfo
next-env.d.ts
```

`.env.example` (copiar para `.env`):

```
DATABASE_URL=postgres://market77:market77@localhost:5432/market77
DATABASE_APP_URL=postgres://market77_app:market77_app@localhost:5432/market77
```

[ ] Passo 3: Criar `docker-compose.yml` e `docker/init.sql`

`docker-compose.yml`:

```
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_USER: market77
      POSTGRES_PASSWORD: market77
      POSTGRES_DB: market77
    ports: ["5432:5432"]
    volumes:
      - pgdata:/var/lib/postgresql/data
      - ./docker/init.sql:/docker-entrypoint-initdb.d/init.sql
volumes:
  pgdata: {}
```

`docker/init.sql`:

```
CREATE DATABASE market77_test;
```

[ ] Passo 4: Criar `src/db/client.ts`

```
import postgres, { type Sql } from "postgres";

function required(name: string): string {
  const v = process.env[name];
  if (!v) throw new Error(`Missing env var ${name}`);
  return v;
}
// Cached on globalThis so Next dev hot reload does not open new pools every edit.
const g = globalThis as unknown as { __m77?: { admin: Sql; app: Sql } };
g.__m77 ??= {
  admin: postgres(required("DATABASE_URL"), { max: 5 }),
  app: postgres(required("DATABASE_APP_URL"), { max: 10 }),
};

/** Owner connection: bypasses RLS. Only auth code and migrations may use it. */
export const adminSql = g.__m77.admin;
/** Application role: subject to RLS. All business data goes through withOrg(). */
export const appSql = g.__m77.app;
```

[ ] Passo 5: Criar `src/db/migrate.ts` e `scripts/migrate.ts`

`src/db/migrate.ts`:

```
import { readdir, readFile } from "node:fs/promises";
import path from "node:path";
import type { Sql } from "postgres";

const DIR = path.join(process.cwd(), "src/db/migrations");

/** Applies pending .sql files in name order. Returns the names applied. */
export async function migrate(sql: Sql): Promise<string[]> {
  await sql`create table if not exists schema_migrations (
    name text primary key, applied_at timestamptz not null default now())`;
  const done = new Set((await sql`select name from schema_migrations`).map((r) => r.name as string));
  const files = (await readdir(DIR)).filter((f) => f.endsWith(".sql")).sort();
  const applied: string[] = [];
  for (const file of files) {
    if (done.has(file)) continue;
    const text = await readFile(path.join(DIR, file), "utf8");
```

```
    await sql.begin(async (tx) => {
      await tx.unsafe(text);
      await tx`insert into schema_migrations (name) values (${file})`;
    });
    applied.push(file);
  }
  return applied;
}
```

`scripts/migrate.ts`:

```
import { adminSql } from "../src/db/client";
import { migrate } from "../src/db/migrate";

async function main() {
  const applied = await migrate(adminSql);
  console.log(applied.length ? `applied: ${applied.join(", ")}` : "up to date");
  await adminSql.end();
}
main();
```

Criar também o arquivo vazio `src/db/migrations/.gitkeep`.

[ ] Passo 6: Criar `vitest.config.ts`, `tests/global-setup.ts`, `tests/helpers.ts`

`vitest.config.ts`:

```
import path from "node:path";
import { defineConfig } from "vitest/config";

const env = {
  DATABASE_URL: "postgres://market77:market77@localhost:5432/market77_test",
  DATABASE_APP_URL: "postgres://market77_app:market77_app@localhost:5432/market77_test",
};
Object.assign(process.env, env);
export default defineConfig({
  resolve: { alias: { "@": path.resolve("src") } },
  test: {
    env,
    globalSetup: ["tests/global-setup.ts"],
    fileParallelism: false, // tests share one database
    hookTimeout: 30000,
  },
});
```

`tests/global-setup.ts`:

```
import postgres from "postgres";
import { migrate } from "../src/db/migrate";

export default async function setup() {
  const sql = postgres(process.env.DATABASE_URL!, { max: 1 });
  await migrate(sql);
  await sql.end();
}
```

`tests/helpers.ts`:

```
import { adminSql } from "@/db/client";

const TABLES = [
  "stock_movements", "market_products", "product_barcodes", "products",
  "markets", "memberships", "sessions", "users", "organizations",
];

export async function resetDb(): Promise<void> {
```

```
  await adminSql.unsafe(`truncate ${TABLES.join(", ")} restart identity cascade`);
}

export async function makeOrg(name = "Org"): Promise<string> {
  const [r] = await adminSql`insert into organizations (name) values (${name}) returning id`;
  return r.id as string;
}
```

Observação: `resetDb` só funciona depois que a migration da Tarefa 2 existir; a Tarefa 1 não o chama.

[ ] Passo 7: Escrever o teste de fumaça `tests/db.test.ts`

```
import { describe, expect, it } from "vitest";
import { adminSql } from "@/db/client";

describe("database", () => {
  it("connects to the test database", async () => {
    const [r] = await adminSql`select current_database() as db`;
    expect(r.db).toBe("market77_test");
  });
});
```

[ ] Passo 8: Subir o banco, instalar e rodar

Executar: `docker compose up -d && npm install && npm test`

Esperado: PASS (1 teste). Se falhar com `database "market77_test" does not exist`, o volume já existia sem o `init.sql`: rode `docker compose down -v && docker compose up -d` e repita.

[ ] Passo 9: Commit

```
git add -A
git commit -m "chore: scaffold project, test database and migration runner"
```

TAREFA 2

Schema, RLS e `withOrg`

Arquivos:

- Criar: `src/db/migrations/0001_init.sql`, `src/db/tenant.ts`
- Modificar: `tests/helpers.ts` (adiciona `seedMarketProduct`)
- Teste: `tests/db/rls.test.ts`

Interfaces:

- Consome: `adminSql`, `appSql`, `resetDb`, `makeOrg` (Tarefa 1).
- Produz: `withOrg<T>(orgId: string, fn: (tx: Tx) => Promise<T>): Promise<T>` e `type Tx` (de `@/db/tenant`); `seedMarketProduct(orgId, priceCents?)` retornando `{ marketId, productId, marketProductId }` (de `tests/helpers`). Tabelas conforme a spec seção 4 (sem lotes, terminais e vendas).

[ ] Passo 1: Criar `src/db/tenant.ts`

```
import type { TransactionSql } from "postgres";
import { appSql } from "./client";

export type Tx = TransactionSql;

/**
 * Runs fn in a transaction scoped to one organization. RLS policies read
 * app.org_id, so every query inside fn only sees (and writes) that tenant.
 */
export async function withOrg<T>(orgId: string, fn: (tx: Tx) => Promise<T>): Promise<T> {
  const result = await appSql.begin(async (tx) => {
    await tx`select set_config('app.org_id', ${orgId}, true)`;
    return fn(tx);
  });
  return result as unknown as T;
}
```

[ ] Passo 2: Acrescentar `seedMarketProduct` em `tests/helpers.ts`

```
import { withOrg } from "@/db/tenant";

export async function seedMarketProduct(orgId: string, priceCents = 500) {
  return withOrg(orgId, async (tx) => {
    const [m] = await tx`insert into markets (org_id, name) values (${orgId}, 'Loja') returning id`;
    const [p] = await tx`insert into products (org_id, name) values (${orgId}, 'Água') returning id`;
    const [mp] = await tx`
      insert into market_products (org_id, market_id, product_id, price_cents)
      values (${orgId}, ${m.id}, ${p.id}, ${priceCents}) returning id`;
    return {
      marketId: m.id as string,
      productId: p.id as string,
      marketProductId: mp.id as string,
    };
  });
}
```

(O `import` vai junto dos imports existentes no topo do arquivo.)

[ ] Passo 3: Escrever os testes `tests/db/rls.test.ts`

```
import postgres from "postgres";
import { beforeEach, describe, expect, it } from "vitest";
import { adminSql } from "@/db/client";
import { withOrg } from "@/db/tenant";
import { makeOrg, resetDb, seedMarketProduct } from "../helpers";

beforeEach(resetDb);
describe("row level security", () => {
  it("isolates markets per tenant", async () => {
    const a = await makeOrg("A");
    const b = await makeOrg("B");
    await withOrg(a, (tx) => tx`insert into markets (org_id, name) values (${a}, 'Loja A')`);
    await withOrg(b, (tx) => tx`insert into markets (org_id, name) values (${b}, 'Loja B')`);
    const seen = await withOrg(a, (tx) => tx`select name from markets`);
    expect(seen.map((r) => r.name)).toEqual(["Loja A"]);
  });

  it("rejects writing a row that belongs to another org", async () => {
    const a = await makeOrg("A");
    const b = await makeOrg("B");
    await expect(
      withOrg(a, (tx) => tx`insert into markets (org_id, name) values (${b}, 'x')`),
    ).rejects.toThrow(/row-level security/);
  });

  it("sees nothing, without error, on a reused connection after withOrg", async () => {
    const a = await makeOrg("A");
    await adminSql`insert into markets (org_id, name) values (${a}, 'Loja A')`;
    // single-connection pool guarantees the same connection serves both calls
    const pool = postgres(process.env.DATABASE_APP_URL!, { max: 1 });
    try {
      await pool.begin(async (tx) => {
        await tx`select set_config('app.org_id', ${a}, true)`;
        const rows = await tx`select * from markets`;
        expect(rows).toHaveLength(1);
      });
      const afterwards = await pool`select * from markets`;
      expect(afterwards).toHaveLength(0);
    } finally {
      await pool.end();
    }
  });

  it("blocks a foreign key that points at another tenant's row", async () => {
    const a = await makeOrg("A");
    const b = await makeOrg("B");
    const theirs = await seedMarketProduct(b);
    const mine = await seedMarketProduct(a);
    await expect(
      withOrg(a, (tx) => tx`
        insert into market_products (org_id, market_id, product_id, price_cents)
        values (${a}, ${theirs.marketId}, ${mine.productId}, 100)`),
    ).rejects.toThrow(/foreign key/);
  });
  it("keeps stock_movements append-only for the app role", async () => {
    const a = await makeOrg("A");
    const { marketProductId } = await seedMarketProduct(a);
    await withOrg(a, (tx) => tx`
      insert into stock_movements (org_id, market_product_id, type, quantity)
      values (${a}, ${marketProductId}, 'in', 5)`);
    await expect(withOrg(a, (tx) => tx`update stock_movements set quantity = 1`)).rejects.toThrow(
      /permission denied/,
    );
    await expect(withOrg(a, (tx) => tx`delete from stock_movements`)).rejects.toThrow(
      /permission denied/,
    );
  });

  it("enforces the sign of each movement type", async () => {
    const a = await makeOrg("A");
```

```
    const { marketProductId } = await seedMarketProduct(a);
    const insert = (type: string, quantity: number) =>
      withOrg(a, (tx) => tx`
        insert into stock_movements (org_id, market_product_id, type, quantity)
        values (${a}, ${marketProductId}, ${type}, ${quantity})`);
    await expect(insert("in", -1)).rejects.toThrow(/check constraint/);
    await expect(insert("sale", 1)).rejects.toThrow(/check constraint/);
    await expect(insert("adjustment", -3)).resolves.toBeDefined();
  });
});
```

[ ] Passo 4: Rodar e ver falhar

Executar: `npm test -- tests/db/rls.test.ts`

Esperado: FAIL (`relation "markets" does not exist`).

[ ] Passo 5: Criar `src/db/migrations/0001_init.sql`

```
-- Application role. Dev password only: production must run
-- ALTER ROLE market77_app PASSWORD '...' with a real secret.
DO $$
BEGIN
  IF NOT EXISTS (SELECT FROM pg_roles WHERE rolname = 'market77_app') THEN
    CREATE ROLE market77_app LOGIN PASSWORD 'market77_app' NOSUPERUSER NOBYPASSRLS;
  END IF;
END $$;
-- Auth tables: no RLS, only reached through the admin connection (src/auth).
CREATE TABLE organizations (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name text NOT NULL,
  created_at timestamptz NOT NULL DEFAULT now()
);
CREATE TABLE users (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  email text NOT NULL UNIQUE,
  password_hash text NOT NULL,
  created_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE memberships (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  org_id uuid NOT NULL REFERENCES organizations (id) ON DELETE CASCADE,
  user_id uuid NOT NULL REFERENCES users (id) ON DELETE CASCADE,
  role text NOT NULL CHECK (role IN ('owner', 'manager', 'restocker')),
  UNIQUE (org_id, user_id)
);

CREATE TABLE sessions (
  token_hash text PRIMARY KEY,
  user_id uuid NOT NULL REFERENCES users (id) ON DELETE CASCADE,
  expires_at timestamptz NOT NULL
);

-- Business tables: org_id on every row; composite FKs keep children in the same tenant.
CREATE TABLE markets (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  org_id uuid NOT NULL REFERENCES organizations (id) ON DELETE CASCADE,
  name text NOT NULL,
  address text,
  timezone text NOT NULL DEFAULT 'America/Sao_Paulo',
  cnpj text,
  created_at timestamptz NOT NULL DEFAULT now(),
  UNIQUE (id, org_id)
);
CREATE TABLE products (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  org_id uuid NOT NULL REFERENCES organizations (id) ON DELETE CASCADE,
  name text NOT NULL,
  category text,
  created_at timestamptz NOT NULL DEFAULT now(),
  UNIQUE (id, org_id)
);

CREATE TABLE product_barcodes (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  org_id uuid NOT NULL,
  product_id uuid NOT NULL,
  code text NOT NULL,
  UNIQUE (org_id, code),
  FOREIGN KEY (product_id, org_id) REFERENCES products (id, org_id) ON DELETE CASCADE
);

CREATE TABLE market_products (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  org_id uuid NOT NULL,
  market_id uuid NOT NULL,
  product_id uuid NOT NULL,
```

```
  price_cents integer NOT NULL CHECK (price_cents > 0),
  min_stock integer NOT NULL DEFAULT 0 CHECK (min_stock >= 0),
  active boolean NOT NULL DEFAULT true,
  UNIQUE (market_id, product_id),
  UNIQUE (id, org_id),
  FOREIGN KEY (market_id, org_id) REFERENCES markets (id, org_id),
  FOREIGN KEY (product_id, org_id) REFERENCES products (id, org_id)
);
CREATE TABLE stock_movements (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  org_id uuid NOT NULL,
  market_product_id uuid NOT NULL,
  type text NOT NULL CHECK (type IN ('in', 'sale', 'loss', 'adjustment')),
  quantity integer NOT NULL,
  note text,
  created_by uuid REFERENCES users (id),
  created_at timestamptz NOT NULL DEFAULT now(),
  CHECK (
    (type = 'in' AND quantity > 0)
    OR (type IN ('sale', 'loss') AND quantity < 0)
    OR (type = 'adjustment' AND quantity <> 0)
  ),
  FOREIGN KEY (market_product_id, org_id) REFERENCES market_products (id, org_id)
);
CREATE INDEX stock_movements_mp_idx ON stock_movements (market_product_id);

-- Tenant isolation. nullif() because a reused pooled connection keeps app.org_id
-- as '' (not NULL) after SET LOCAL ends, and ''::uuid would raise an error.
DO $$
DECLARE t text;
BEGIN
  FOREACH t IN ARRAY ARRAY['markets', 'products', 'product_barcodes', 'market_products',
      'stock_movements']
  LOOP
    EXECUTE format('ALTER TABLE %I ENABLE ROW LEVEL SECURITY', t);
    EXECUTE format($p$
      CREATE POLICY tenant_isolation ON %I
        USING (org_id = nullif(current_setting('app.org_id', true), '')::uuid)
        WITH CHECK (org_id = nullif(current_setting('app.org_id', true), '')::uuid)$p$, t);
  END LOOP;
END $$;

GRANT SELECT, INSERT, UPDATE, DELETE ON markets, products, product_barcodes, market_products TO
    market77_app;
-- Ledger is append-only for the app role.
GRANT SELECT, INSERT ON stock_movements TO market77_app;
```

[ ] Passo 6: Rodar e ver passar

Executar: `npm test`

Esperado: PASS (todos os testes, incluindo os 6 de `rls.test.ts`).

[ ] Passo 7: Commit

```
git add -A
git commit -m "feat: add multi-tenant schema with RLS and withOrg helper"
```

TAREFA 3

Utilitários, senhas e sessões

Arquivos:

- Criar: `src/lib/errors.ts`, `src/lib/validate.ts`, `src/lib/money.ts`, `src/lib/messages.ts`
- Criar: `src/auth/password.ts`, `src/auth/sessions.ts`
- Modificar: `tests/helpers.ts` (adiciona `makeUser`)
- Teste: `tests/lib/validate.test.ts`, `tests/lib/money.test.ts`, `tests/auth/password.test.ts`, `tests/auth/sessions.test.ts`

Interfaces:

- Produz:
  - `class AppError extends Error { code: string }`, `isUniqueViolation(e)`, `isForeignKeyViolation(e)` (`@/lib/errors`)
  - `parse<S extends z.ZodTypeAny>(schema: S, input: unknown): z.output<S>` (lança `AppError("invalid_input")`), `formToObject(fd: FormData): Record<string, string>` (`@/lib/validate`)
  - `parseBRL(input: string): number | null`, `formatBRL(cents: number): string` (`@/lib/money`)
  - `message(code: string): string` (`@/lib/messages`)
  - `hashPassword(pw): Promise<string>`, `verifyPassword(pw, stored): Promise<boolean>` (`@/auth/password`)
  - `createSession(userId): Promise<{ token: string; expiresAt: Date }>`, `getSessionUser(token): Promise<{ userId: string } | null>`, `deleteSession(token): Promise<void>` (`@/auth/sessions`)
  - `makeUser(email: string): Promise<string>` (`tests/helpers`)

[ ] Passo 1: Escrever os testes

`tests/lib/validate.test.ts`:

```
import { describe, expect, it } from "vitest";
import { z } from "zod";
import { AppError } from "@/lib/errors";
import { formToObject, parse } from "@/lib/validate";

describe("parse", () => {
  it("returns the parsed output", () => {
    expect(parse(z.object({ n: z.coerce.number() }), { n: "3" })).toEqual({ n: 3 });
  });

  it("throws AppError invalid_input", () => {
    let err: unknown;
    try {
      parse(z.object({ n: z.number() }), { n: "x" });
    } catch (e) {
      err = e;
```

```
    }
    expect(err).toBeInstanceOf(AppError);
    expect((err as AppError).code).toBe("invalid_input");
  });
});

describe("formToObject", () => {
  it("keeps non-empty string fields and drops blanks and files", () => {
    const fd = new FormData();
    fd.set("name", " Ana ");
    fd.set("address", "   ");
    fd.set("file", new Blob(["x"]), "x.txt");
    expect(formToObject(fd)).toEqual({ name: " Ana " });
  });
});
```

tests/lib/money.test.ts:

```
import { describe, expect, it } from "vitest";
import { formatBRL, parseBRL } from "@/lib/money";

describe("parseBRL", () => {
  it.each([
    ["5,90", 590],
    ["5.9", 590],
    ["10", 1000],
    ["R$ 0,05", 5],
    ["0.10", 10],
  ])("parses %s", (input, cents) => expect(parseBRL(input)).toBe(cents));

  it.each(["", "abc", "1,234", "-1", "1.234,50", "5,"])("rejects %j", (input) =>
    expect(parseBRL(input)).toBeNull(),
  );
});
describe("formatBRL", () => {
  it("formats cents as reais", () => {
    expect(formatBRL(590).replace(/\s/g, " ")).toBe("R$ 5,90");
  });
});
```

tests/auth/password.test.ts:

```
import { describe, expect, it } from "vitest";
import { hashPassword, verifyPassword } from "@/auth/password";

describe("password", () => {
  it("verifies the right password and rejects the wrong one", async () => {
    const stored = await hashPassword("segredo-123");
    expect(await verifyPassword("segredo-123", stored)).toBe(true);
    expect(await verifyPassword("outra-senha", stored)).toBe(false);
  });

  it("uses a random salt", async () => {
    expect(await hashPassword("x1234567")).not.toBe(await hashPassword("x1234567"));
  });

  it("rejects malformed stored values instead of throwing", async () => {
    expect(await verifyPassword("x", "lixo")).toBe(false);
    expect(await verifyPassword("x", "")).toBe(false);
  });
});
```

tests/auth/sessions.test.ts:

```
import { beforeEach, describe, expect, it } from "vitest";
import { adminSql } from "@/db/client";
import { createSession, deleteSession, getSessionUser } from "@/auth/sessions";
import { makeUser, resetDb } from "../helpers";

beforeEach(resetDb);

describe("sessions", () => {
  it("resolves a valid token to its user", async () => {
    const userId = await makeUser("a@x.com");
    const { token } = await createSession(userId);
    expect(await getSessionUser(token)).toEqual({ userId });
  });

  it("stores only a hash of the token", async () => {
    const userId = await makeUser("a@x.com");
    const { token } = await createSession(userId);
    const rows = await adminSql`select token_hash from sessions`;
    expect(rows[0].token_hash).not.toBe(token);
  });

  it("rejects expired, deleted and unknown tokens", async () => {
    const userId = await makeUser("a@x.com");
    const { token } = await createSession(userId);
    await adminSql`update sessions set expires_at = now() - interval '1 minute'`;
    expect(await getSessionUser(token)).toBeNull();

    const second = await createSession(userId);
    await deleteSession(second.token);
    expect(await getSessionUser(second.token)).toBeNull();
    expect(await getSessionUser("nao-existe")).toBeNull();
  });
});
```

[ ] Passo 2: Adicionar `makeUser` em `tests/helpers.ts`

```
export async function makeUser(email: string): Promise<string> {
  const [r] = await adminSql`
    insert into users (email, password_hash) values (${email}, 'x') returning id`;
  return r.id as string;
}
```

[ ] Passo 3: Rodar e ver falhar

Executar: `npm test -- tests/lib tests/auth`

Esperado: FAIL (módulos não existem).

[ ] Passo 4: Implementar `src/lib/*`

`src/lib/errors.ts`:

```
export class AppError extends Error {
  constructor(
    public readonly code: string,
    message?: string,
  ) {
    super(message ?? code);
    this.name = "AppError";
  }
}

const pgCode = (e: unknown) => (e as { code?: string } | null)?.code;
export const isUniqueViolation = (e: unknown) => pgCode(e) === "23505";
export const isForeignKeyViolation = (e: unknown) => pgCode(e) === "23503";
```

`src/lib/validate.ts`:

```
import type { z } from "zod";
import { AppError } from "./errors";

export function parse<S extends z.ZodTypeAny>(schema: S, input: unknown): z.output<S> {
  const r = schema.safeParse(input);
  if (!r.success) throw new AppError("invalid_input", r.error.issues[0]?.message);
  return r.data;
}
/** FormData to plain object; blank fields are dropped so optional zod fields work. */
export function formToObject(fd: FormData): Record<string, string> {
  const out: Record<string, string> = {};
  for (const [k, v] of fd.entries()) {
    if (typeof v === "string" && v.trim() !== "") out[k] = v;
  }
  return out;
}
```

`src/lib/money.ts`:

```
/** "5,90" | "5.9" | "R$ 10" to cents. No thousands separator support. Null if invalid. */
export function parseBRL(input: string): number | null {
  const s = input.trim().replace(/^R\$\s*/, "");
  if (!/^\d+([.,]\d{1,2})?$/.test(s)) return null;
  const [int, dec = ""] = s.split(/[.,]/);
  return Number(int) * 100 + Number(dec.padEnd(2, "0"));
}
export const formatBRL = (cents: number): string =>
  (cents / 100).toLocaleString("pt-BR", { style: "currency", currency: "BRL" });
```

`src/lib/messages.ts`:

```
const MESSAGES: Record<string, string> = {
  invalid_input: "Dados inválidos. Confira os campos.",
  email_taken: "Já existe uma conta com este e-mail.",
  invalid_credentials: "E-mail ou senha incorretos.",
  barcode_taken: "Este código de barras já está em uso.",
  not_found: "Item não encontrado.",
  already_member: "Este usuário já faz parte da organização.",
  forbidden: "Você não tem permissão para isso.",
};

export const message = (code: string): string => MESSAGES[code] ?? "Erro inesperado.";
```

[ ] Passo 5: Implementar `src/auth/password.ts` e `src/auth/sessions.ts`

`src/auth/password.ts`:

```
import { randomBytes, scrypt as scryptCb, timingSafeEqual } from "node:crypto";
import { promisify } from "node:util";

const scrypt = promisify(scryptCb) as (pw: string, salt: Buffer, len: number) => Promise<Buffer>;

export async function hashPassword(pw: string): Promise<string> {
  const salt = randomBytes(16);
  const key = await scrypt(pw, salt, 64);
  return `scrypt$${salt.toString("hex")}$${key.toString("hex")}`;
}

export async function verifyPassword(pw: string, stored: string): Promise<boolean> {
  const [alg, saltHex, keyHex] = stored.split("$");
  if (alg !== "scrypt" || !saltHex || !keyHex) return false;
  const expected = Buffer.from(keyHex, "hex");
  const key = await scrypt(pw, Buffer.from(saltHex, "hex"), 64);
  return key.length === expected.length && timingSafeEqual(key, expected);
}
```

`src/auth/sessions.ts`:

```
import { createHash, randomBytes } from "node:crypto";
import { adminSql } from "@/db/client";

const hashToken = (t: string) => createHash("sha256").update(t).digest("hex");

// ponytail: expired rows are never purged, add a cleanup job if the table grows.
export async function createSession(userId: string): Promise<{ token: string; expiresAt: Date }> {
  const token = randomBytes(32).toString("base64url");
  const [row] = await adminSql`
    insert into sessions (token_hash, user_id, expires_at)
    values (${hashToken(token)}, ${userId}, now() + interval '30 days')
    returning expires_at`;
  return { token, expiresAt: row.expires_at as Date };
}

export async function getSessionUser(token: string): Promise<{ userId: string } | null> {
  const [row] = await adminSql`
    select user_id from sessions where token_hash = ${hashToken(token)} and expires_at > now()`;
  return row ? { userId: row.user_id as string } : null;
}

export async function deleteSession(token: string): Promise<void> {
  await adminSql`delete from sessions where token_hash = ${hashToken(token)}`;
}
```

[ ] Passo 6: Rodar e ver passar

Executar: `npm test`

Esperado: PASS.

[ ] Passo 7: Commit

```
git add -A
git commit -m "feat: add validation helpers, money parsing, passwords and sessions"
```

TAREFA 4

Permissões

Arquivos:

- Criar: `src/auth/permissions.ts`
- Teste: `tests/auth/permissions.test.ts`

Interfaces:

- Produz: `ROLES`, `type Role = "owner" | "manager" | "restocker"`, `type Action` (`"member:write" | "market:write" | "product:write" | "price:write" | "stock:move" | "stock:read" | "catalog:read"`), `can(role: Role, action: Action): boolean` (`@/auth/permissions`).

[ ] Passo 1: Escrever o teste

```
import { describe, expect, it } from "vitest";
import { can } from "@/auth/permissions";

describe("can", () => {
  it("lets the owner do everything", () => {
    for (const a of ["member:write", "market:write", "product:write", "price:write", "stock:move"] as
        const) {
      expect(can("owner", a)).toBe(true);
    }
  });

  it("lets the manager manage catalog and stock but not markets or members", () => {
    expect(can("manager", "product:write")).toBe(true);
    expect(can("manager", "price:write")).toBe(true);
    expect(can("manager", "market:write")).toBe(false);
    expect(can("manager", "member:write")).toBe(false);
  });

  it("lets the restocker move stock only", () => {
    expect(can("restocker", "stock:move")).toBe(true);
    expect(can("restocker", "stock:read")).toBe(true);
    expect(can("restocker", "price:write")).toBe(false);
    expect(can("restocker", "product:write")).toBe(false);
  });
});
```

[ ] Passo 2: Rodar e ver falhar

Executar: `npm test -- tests/auth/permissions.test.ts`

Esperado: FAIL (módulo não existe).

[ ] Passo 3: Implementar `src/auth/permissions.ts`

```
export const ROLES = ["owner", "manager", "restocker"] as const;
export type Role = (typeof ROLES)[number];

const MATRIX = {
  owner: ["member:write", "market:write", "product:write", "price:write", "stock:move", "stock:read",
      "catalog:read"],
  manager: ["product:write", "price:write", "stock:move", "stock:read", "catalog:read"],
  restocker: ["stock:move", "stock:read", "catalog:read"],
} as const;

export type Action = (typeof MATRIX)["owner"][number];

export const can = (role: Role, action: Action): boolean =>
  (MATRIX[role] as readonly Action[]).includes(action);
```

[ ] Passo 4: Rodar e ver passar

Executar: `npm test -- tests/auth/permissions.test.ts`

Esperado: PASS.

[ ] Passo 5: Commit

```
git add -A
git commit -m "feat: add role permission matrix"
```

TAREFA 5

Serviço de autenticação e guard

Arquivos:

- Criar: `src/auth/service.ts`, `src/auth/guard.ts`
- Teste: `tests/auth/service.test.ts`

Interfaces:

- Consome: `adminSql`, `hashPassword`, `verifyPassword`, `createSession`, `getSessionUser`, `parse`, `AppError`, `isUniqueViolation`, `Role`, `can`, `Action`.
- Produces (`@/auth/service`):
  - `signUp(raw): Promise<{ userId; orgId; token; expiresAt }>` com `raw = { email, password, orgName }`
  - `login(raw): Promise<{ userId; token; expiresAt }>` com `raw = { email, password }`
  - `addMember(orgId: string, raw): Promise<void>` com `raw = { email, password?, role: "manager" | "restocker" }`
  - `listMemberships(userId): Promise<{ orgId; orgName; role: Role }[]>`
  - `getMembership(userId, orgId): Promise<Role | null>` (devolve `null` se `orgId` não for UUID)
- Produces (`@/auth/guard`): `SESSION_COOKIE = "m77_session"`, `requireUser(): Promise<{ userId }>`, `requireOrg(orgId, action?): Promise<{ userId; role: Role }>`.

[ ] Passo 1: Escrever os testes `tests/auth/service.test.ts`

```
import { beforeEach, describe, expect, it } from "vitest";
import { adminSql } from "@/db/client";
import { addMember, getMembership, listMemberships, login, signUp } from "@/auth/service";
import { getSessionUser } from "@/auth/sessions";
import { AppError } from "@/lib/errors";
import { resetDb } from "../helpers";

beforeEach(resetDb);
const ana = { email: "ana@x.com", password: "senha-forte-1", orgName: "Mercados Ana" };

async function codeOf(p: Promise<unknown>): Promise<string | undefined> {
  try {
    await p;
  } catch (e) {
    return e instanceof AppError ? e.code : "not-app-error";
  }
}

describe("signUp", () => {
  it("creates user, organization, owner membership and a session", async () => {
    const r = await signUp(ana);
    expect(await getSessionUser(r.token)).toEqual({ userId: r.userId });
    expect(await getMembership(r.userId, r.orgId)).toBe("owner");
  });

  it("treats emails case- and whitespace-insensitively", async () => {
    await signUp(ana);
    expect(await codeOf(signUp({ ...ana, email: "  ANA@x.com " }))).toBe("email_taken");
    const [{ count }] = await adminSql`select count(*)::int as count from users`;
    expect(count).toBe(1);
  });
  it("rejects short passwords and bad emails", async () => {
    expect(await codeOf(signUp({ ...ana, password: "1234567" }))).toBe("invalid_input");
    expect(await codeOf(signUp({ ...ana, email: "nope" }))).toBe("invalid_input");
  });
});

describe("login", () => {
  it("logs in with the right password", async () => {
    const s = await signUp(ana);
    const r = await login({ email: "ANA@x.com", password: ana.password });
    expect(r.userId).toBe(s.userId);
  });

  it("gives the same error for wrong password and unknown email", async () => {
    await signUp(ana);
    expect(await codeOf(login({ email: ana.email, password: "errada-123"
        }))).toBe("invalid_credentials");
    expect(await codeOf(login({ email: "x@x.com", password: "qualquer-123"
        }))).toBe("invalid_credentials");
  });
});

describe("memberships", () => {
  it("adds a new user as manager and lets an existing user join a second org", async () => {
    const a = await signUp(ana);
    await addMember(a.orgId, { email: "gil@x.com", password: "senha-gil-123", role: "manager" });
    const gil = await login({ email: "gil@x.com", password: "senha-gil-123" });
    expect(await getMembership(gil.userId, a.orgId)).toBe("manager");

    const b = await signUp({ email: "bia@x.com", password: "senha-bia-123", orgName: "Bia" });
    await addMember(b.orgId, { email: "gil@x.com", role: "restocker" });
    const orgs = await listMemberships(gil.userId);
    expect(orgs.map((o) => [o.orgName, o.role]).sort()).toEqual([
      ["Bia", "restocker"],
      ["Mercados Ana", "manager"],
    ]);
  });

  it("requires a password for a brand new user and rejects duplicates", async () => {
```

```
    const a = await signUp(ana);
    expect(await codeOf(addMember(a.orgId, { email: "novo@x.com", role: "manager"
        }))).toBe("invalid_input");
    await addMember(a.orgId, { email: "gil@x.com", password: "senha-gil-123", role: "manager" });
    expect(await codeOf(addMember(a.orgId, { email: "gil@x.com", role: "manager"
        }))).toBe("already_member");
  });
  it("returns null for a non-uuid org id instead of throwing", async () => {
    const a = await signUp(ana);
    expect(await getMembership(a.userId, "not-a-uuid")).toBeNull();
  });
});
```

[ ] Passo 2: Rodar e ver falhar

Executar: `npm test -- tests/auth/service.test.ts`

Esperado: FAIL (módulo não existe).

[ ] Passo 3: Implementar `src/auth/service.ts`

```
import { z } from "zod";
import { adminSql } from "@/db/client";
import { AppError, isUniqueViolation } from "@/lib/errors";
import { parse } from "@/lib/validate";
import { hashPassword, verifyPassword } from "./password";
import type { Role } from "./permissions";
import { createSession } from "./sessions";
const email = z.string().trim().toLowerCase().email();
const password = z.string().min(8).max(200);

const SignUp = z.object({ email, password, orgName: z.string().trim().min(2).max(100) });
const Login = z.object({ email, password: z.string().min(1) });
const AddMember = z.object({
  email,
  password: password.optional(),
  role: z.enum(["manager", "restocker"]),
});

export async function signUp(raw: unknown) {
  const input = parse(SignUp, raw);
  const passwordHash = await hashPassword(input.password);
  let ids: { userId: string; orgId: string };
  try {
    ids = await adminSql.begin(async (tx) => {
      const [u] = await tx`
        insert into users (email, password_hash) values (${input.email}, ${passwordHash}) returning id`;
      const [o] = await tx`insert into organizations (name) values (${input.orgName}) returning id`;
      await tx`insert into memberships (org_id, user_id, role) values (${o.id}, ${u.id}, 'owner')`;
      return { userId: u.id as string, orgId: o.id as string };
    });
  } catch (e) {
    if (isUniqueViolation(e)) throw new AppError("email_taken");
    throw e;
  }
  const { token, expiresAt } = await createSession(ids.userId);
  return { ...ids, token, expiresAt };
}

// ponytail: unknown email skips the scrypt cost, so timing can reveal which emails exist.
// Verify against a dummy hash if account enumeration becomes a concern.
export async function login(raw: unknown) {
  const input = parse(Login, raw);
  const [u] = await adminSql`select id, password_hash from users where email = ${input.email}`;
  const ok = u ? await verifyPassword(input.password, u.password_hash as string) : false;
  if (!u || !ok) throw new AppError("invalid_credentials");
  const { token, expiresAt } = await createSession(u.id as string);
  return { userId: u.id as string, token, expiresAt };
}

/** Existing users keep their own password; only brand new users get the one provided. */
export async function addMember(orgId: string, raw: unknown): Promise<void> {
  const input = parse(AddMember, raw);
  await adminSql.begin(async (tx) => {
    let [u] = await tx`select id from users where email = ${input.email}`;
    if (!u) {
      if (!input.password) throw new AppError("invalid_input", "password required for new user");
      const hash = await hashPassword(input.password);
      [u] = await tx`insert into users (email, password_hash) values (${input.email}, ${hash}) returning
           id`;
    }
    try {
      await tx`insert into memberships (org_id, user_id, role) values (${orgId}, ${u.id},
          ${input.role})`;
    } catch (e) {
      if (isUniqueViolation(e)) throw new AppError("already_member");
      throw e;
    }
  });
}

export async function listMemberships(userId: string) {
```

```
  const rows = await adminSql`
    select m.org_id, m.role, o.name
    from memberships m join organizations o on o.id = m.org_id
    where m.user_id = ${userId} order by o.name`;
  return rows.map((r) => ({
    orgId: r.org_id as string,
    orgName: r.name as string,
    role: r.role as Role,
  }));
}

export async function getMembership(userId: string, orgId: string): Promise<Role | null> {
  if (!z.string().uuid().safeParse(orgId).success) return null;
  const [r] = await adminSql`
    select role from memberships where user_id = ${userId} and org_id = ${orgId}`;
  return r ? (r.role as Role) : null;
}
```

[ ] Passo 4: Rodar e ver passar

Executar: `npm test -- tests/auth/service.test.ts`

Esperado: PASS.

[ ] Passo 5: Implementar `src/auth/guard.ts`

O guard depende do runtime do Next (`cookies`, `redirect`) e por isso não tem teste unitário; é exercitado no roteiro manual da Tarefa 10.

```
import { cookies } from "next/headers";
import { redirect } from "next/navigation";
import { AppError } from "@/lib/errors";
import { type Action, type Role, can } from "./permissions";
import { getMembership } from "./service";
import { getSessionUser } from "./sessions";

export const SESSION_COOKIE = "m77_session";
export async function requireUser(): Promise<{ userId: string }> {
  const token = (await cookies()).get(SESSION_COOKIE)?.value;
  const user = token ? await getSessionUser(token) : null;
  if (!user) redirect("/login");
  return user;
}
/** Every page and action must call this: layouts do not re-run on client navigation. */
export async function requireOrg(
  orgId: string,
  action?: Action,
): Promise<{ userId: string; role: Role }> {
  const { userId } = await requireUser();
  const role = await getMembership(userId, orgId);
  if (!role || (action && !can(role, action))) throw new AppError("forbidden");
  return { userId, role };
}
```

[ ] Passo 6: Typecheck e commit

Executar: `npm run typecheck`

Esperado: sem erros.

```
git add -A
git commit -m "feat: add auth service (signup, login, members) and request guard"
```

TAREFA 6

Domínio de mercados e produtos

Arquivos:

- Criar: `src/domain/markets.ts`, `src/domain/products.ts`
- Teste: `tests/domain/markets.test.ts`, `tests/domain/products.test.ts`

Interfaces:

- Consome: `Tx`, `withOrg`, `parse`, `AppError`, `isUniqueViolation`, `makeOrg`, `resetDb`.
- Produz:
  - `createMarket(tx: Tx, orgId: string, raw): Promise<MarketRow>` com `raw = { name, address?, timezone?, cnpj? }`; `listMarkets(tx: Tx): Promise<MarketRow[]>`; `type MarketRow = { id; name; address: string | null; timezone; cnpj: string | null }` (`@/domain/markets`)
  - `createProduct(tx: Tx, orgId: string, raw): Promise<ProductRow>` com `raw = { name, category?, barcodes?: string[] }`; `listProducts(tx: Tx): Promise<ProductRow[]>`; `type ProductRow = { id; name; category: string | null; barcodes: string[] }` (`@/domain/products`)

[ ] Passo 1: Escrever `tests/domain/markets.test.ts`

```
import { beforeEach, describe, expect, it } from "vitest";
import { withOrg } from "@/db/tenant";
import { createMarket, listMarkets } from "@/domain/markets";
import { AppError } from "@/lib/errors";
import { makeOrg, resetDb } from "../helpers";

beforeEach(resetDb);
describe("markets", () => {
  it("creates a market with defaults and lists only the tenant's markets", async () => {
    const a = await makeOrg("A");
    const b = await makeOrg("B");
    const m = await withOrg(a, (tx) => createMarket(tx, a, { name: "Loja Bloco 1" }));
    await withOrg(b, (tx) => createMarket(tx, b, { name: "Loja B" }));
    expect(m.timezone).toBe("America/Sao_Paulo");
    expect((await withOrg(a, (tx) => listMarkets(tx))).map((x) => x.name)).toEqual(["Loja Bloco 1"]);
  });

  it("strips punctuation from the CNPJ and rejects wrong lengths", async () => {
    const a = await makeOrg("A");
    const m = await withOrg(a, (tx) =>
      createMarket(tx, a, { name: "Loja", cnpj: "12.345.678/0001-95" }),
    );
    expect(m.cnpj).toBe("12345678000195");
    await expect(withOrg(a, (tx) => createMarket(tx, a, { name: "Loja", cnpj: "123"
        }))).rejects.toBeInstanceOf(AppError);
  });

  it("rejects short names and unknown time zones", async () => {
    const a = await makeOrg("A");
    await expect(withOrg(a, (tx) => createMarket(tx, a, { name: "x"
        }))).rejects.toBeInstanceOf(AppError);
    await expect(
      withOrg(a, (tx) => createMarket(tx, a, { name: "Loja", timezone: "Marte/Olympus" })),
    ).rejects.toBeInstanceOf(AppError);
  });
});
```

[ ] Passo 2: Escrever `tests/domain/products.test.ts`

```
import { beforeEach, describe, expect, it } from "vitest";
import { withOrg } from "@/db/tenant";
import { createProduct, listProducts } from "@/domain/products";
import { AppError } from "@/lib/errors";
import { makeOrg, resetDb } from "../helpers";

beforeEach(resetDb);
const agua = { name: "Água 500ml", category: "Bebidas", barcodes: ["7891000100103"] };

describe("products", () => {
  it("creates a product with barcodes and lists it", async () => {
    const a = await makeOrg("A");
    await withOrg(a, (tx) => createProduct(tx, a, { ...agua, barcodes: ["7891000100103", "AGUA-01"] }));
    const [p] = await withOrg(a, (tx) => listProducts(tx));
    expect(p.name).toBe("Água 500ml");
    expect(p.barcodes).toEqual(["7891000100103", "AGUA-01"]);
  });

  it("dedupes repeated barcodes inside one request", async () => {
    const a = await makeOrg("A");
    const p = await withOrg(a, (tx) => createProduct(tx, a, { name: "Suco", barcodes: ["12345678",
        "12345678"] }));
    expect(p.barcodes).toEqual(["12345678"]);
  });

  it("rejects a barcode already used in the org and rolls the product back", async () => {
    const a = await makeOrg("A");
    await withOrg(a, (tx) => createProduct(tx, a, agua));
    const err = await withOrg(a, (tx) => createProduct(tx, a, { name: "Outra água", barcodes:
        agua.barcodes })).catch((e) => e);
    expect(err).toBeInstanceOf(AppError);
    expect((err as AppError).code).toBe("barcode_taken");
    expect((await withOrg(a, (tx) => listProducts(tx))).map((p) => p.name)).toEqual(["Água 500ml"]);
  });

  it("allows the same barcode in two different orgs", async () => {
    const a = await makeOrg("A");
    const b = await makeOrg("B");
    await withOrg(a, (tx) => createProduct(tx, a, agua));
    await expect(withOrg(b, (tx) => createProduct(tx, b, agua))).resolves.toBeDefined();
  });

  it("rejects malformed barcodes", async () => {
    const a = await makeOrg("A");
    await expect(
      withOrg(a, (tx) => createProduct(tx, a, { name: "Suco", barcodes: ["ab"] })),
    ).rejects.toBeInstanceOf(AppError);
  });
});
```

[ ] Passo 3: Rodar e ver falhar

Executar: `npm test -- tests/domain`

Esperado: FAIL (módulos não existem).

[ ] Passo 4: Implementar `src/domain/markets.ts`

```
import { z } from "zod";
import type { Tx } from "@/db/tenant";
import { parse } from "@/lib/validate";

const validTimeZone = (tz: string) => {
  try {
    new Intl.DateTimeFormat("en", { timeZone: tz });
    return true;
  } catch {
    return false;
  }
};

const Market = z.object({
  name: z.string().trim().min(2).max(100),
  address: z.string().trim().max(200).optional(),
  timezone: z.string().trim().refine(validTimeZone, "invalid time zone").default("America/Sao_Paulo"),
  // ponytail: length only, no check digits; add CNPJ checksum in the fiscal phase.
  cnpj: z.string().transform((s) => s.replace(/\D/g, "")).pipe(z.string().length(14)).optional(),
});

export type MarketRow = {
  id: string;
  name: string;
  address: string | null;
  timezone: string;
  cnpj: string | null;
};

export async function createMarket(tx: Tx, orgId: string, raw: unknown): Promise<MarketRow> {
  const m = parse(Market, raw);
  const [row] = await tx`
    insert into markets (org_id, name, address, timezone, cnpj)
    values (${orgId}, ${m.name}, ${m.address ?? null}, ${m.timezone}, ${m.cnpj ?? null})
    returning id, name, address, timezone, cnpj`;
  return row as unknown as MarketRow;
}

export async function listMarkets(tx: Tx): Promise<MarketRow[]> {
  const rows = await tx`select id, name, address, timezone, cnpj from markets order by name`;
  return rows as unknown as MarketRow[];
}
```

[ ] Passo 5: Implementar `src/domain/products.ts`

```
import { z } from "zod";
import type { Tx } from "@/db/tenant";
import { AppError, isUniqueViolation } from "@/lib/errors";
import { parse } from "@/lib/validate";

const barcode = z.string().trim().regex(/^[0-9A-Za-z-]{4,32}$/, "invalid barcode");

const Product = z.object({
  name: z.string().trim().min(2).max(150),
  category: z.string().trim().max(60).optional(),
  barcodes: z.array(barcode).max(10).default([]).transform((a) => [...new Set(a)]),
});

export type ProductRow = { id: string; name: string; category: string | null; barcodes: string[] };

export async function createProduct(tx: Tx, orgId: string, raw: unknown): Promise<ProductRow> {
  const p = parse(Product, raw);
  const [row] = await tx`
    insert into products (org_id, name, category)
    values (${orgId}, ${p.name}, ${p.category ?? null}) returning id, name, category`;
  for (const code of p.barcodes) {
    try {
      await tx`insert into product_barcodes (org_id, product_id, code) values (${orgId}, ${row.id},
          ${code})`;
    } catch (e) {
      if (isUniqueViolation(e)) throw new AppError("barcode_taken", code);
      throw e;
    }
  }
  return { id: row.id as string, name: row.name as string, category: row.category as string | null,
      barcodes: p.barcodes };
}
export async function listProducts(tx: Tx): Promise<ProductRow[]> {
  const rows = await tx`
    select p.id, p.name, p.category,
      coalesce(array_agg(b.code order by b.code) filter (where b.code is not null), '{}') as barcodes
    from products p left join product_barcodes b on b.product_id = p.id
    group by p.id order by p.name`;
  return rows as unknown as ProductRow[];
}
```

[ ] Passo 6: Rodar e ver passar

Executar: `npm test`

Esperado: PASS.

[ ] Passo 7: Commit

```
git add -A
git commit -m "feat: add markets and products domain"
```

TAREFA 7

Preço por mercado e estoque

Arquivos:

- Criar: `src/domain/market-products.ts`, `src/domain/stock.ts`
- Teste: `tests/domain/market-products.test.ts`, `tests/domain/stock.test.ts`

Interfaces:

- Consome: `Tx`, `withOrg`, `parse`, `AppError`, `isForeignKeyViolation`, `seedMarketProduct`, `makeOrg`, `resetDb`, `createMarket`, `createProduct`.
- Produz:
  - `setMarketProduct(tx: Tx, orgId: string, raw): Promise<{ id: string }>` com `raw = { marketId, productId, priceCents: number, minStock?: number, active?: boolean }` (upsert por mercado e produto); `listMarketProducts(tx: Tx, marketId: string): Promise<MarketProductRow[]>`; `type MarketProductRow = { id; productId; name; priceCents; minStock; active; balance: number }` (`@/domain/market-products`)
  - `recordMovement(tx: Tx, orgId: string, userId: string | null, raw): Promise<{ id: string; delta: number }>` com `raw = { marketProductId, type: "in" | "sale" | "loss" | "adjustment", quantity: number, note? }`. Para `in`, `sale` e `loss` a quantidade é positiva (o sinal vem do tipo); para `adjustment` é com sinal e diferente de zero. `getBalance(tx, marketProductId): Promise<number>`; `listLowStock(tx, marketId): Promise<MarketProductRow[]>` (ativos com `balance <= minStock`) (`@/domain/stock`)

[ ] Passo 1: Escrever `tests/domain/market-products.test.ts`

```
import { beforeEach, describe, expect, it } from "vitest";
import { withOrg } from "@/db/tenant";
import { listMarketProducts, setMarketProduct } from "@/domain/market-products";
import { AppError } from "@/lib/errors";
import { makeOrg, resetDb, seedMarketProduct } from "../helpers";

beforeEach(resetDb);
const codeOf = async (p: Promise<unknown>) => ((await p.catch((e) => e)) as AppError).code;

describe("market products", () => {
  it("upserts price and min stock for the same market and product", async () => {
    const a = await makeOrg("A");
    const { marketId, productId, marketProductId } = await seedMarketProduct(a, 500);
    const r = await withOrg(a, (tx) => setMarketProduct(tx, a, { marketId, productId, priceCents: 650,
        minStock: 4 }));
    expect(r.id).toBe(marketProductId);
    const [row] = await withOrg(a, (tx) => listMarketProducts(tx, marketId));
    expect([row.priceCents, row.minStock, row.balance]).toEqual([650, 4, 0]);
  });

  it.each([0, -5, 5.5])("rejects price %s", async (priceCents) => {
    const a = await makeOrg("A");
    const { marketId, productId } = await seedMarketProduct(a);
    expect(await codeOf(withOrg(a, (tx) => setMarketProduct(tx, a, { marketId, productId, priceCents
        }))))).toBe("invalid_input");
  });

  it("returns not_found when the market belongs to another tenant", async () => {
    const a = await makeOrg("A");
    const b = await makeOrg("B");
    const mine = await seedMarketProduct(a);
    const theirs = await seedMarketProduct(b);
    expect(
      await codeOf(withOrg(a, (tx) => setMarketProduct(tx, a, { marketId: theirs.marketId, productId:
          mine.productId, priceCents: 100 }))),
    ).toBe("not_found");
  });
});
```

[ ] Passo 2: Escrever `tests/domain/stock.test.ts`

```
import { beforeEach, describe, expect, it } from "vitest";
import { withOrg } from "@/db/tenant";
import { setMarketProduct } from "@/domain/market-products";
import { getBalance, listLowStock, recordMovement } from "@/domain/stock";
import { AppError } from "@/lib/errors";
import { makeOrg, resetDb, seedMarketProduct } from "../helpers";

beforeEach(resetDb);
const codeOf = async (p: Promise<unknown>) => ((await p.catch((e) => e)) as AppError).code;
const move = (orgId: string, marketProductId: string, type: string, quantity: number) =>
  withOrg(orgId, (tx) => recordMovement(tx, orgId, null, { marketProductId, type, quantity }));

describe("stock ledger", () => {
  it("computes the balance from signed movements", async () => {
    const a = await makeOrg("A");
    const { marketProductId: mp } = await seedMarketProduct(a);
    await move(a, mp, "in", 10);
    await move(a, mp, "sale", 3);
    await move(a, mp, "loss", 2);
    expect(await withOrg(a, (tx) => getBalance(tx, mp))).toBe(5);
    await move(a, mp, "adjustment", -4);
    expect(await withOrg(a, (tx) => getBalance(tx, mp))).toBe(1);
  });
  it("rejects negative or zero quantities for in/sale/loss and zero for adjustment", async () => {
    const a = await makeOrg("A");
    const { marketProductId: mp } = await seedMarketProduct(a);
    expect(await codeOf(move(a, mp, "sale", -1))).toBe("invalid_input");
    expect(await codeOf(move(a, mp, "in", 0))).toBe("invalid_input");
    expect(await codeOf(move(a, mp, "adjustment", 0))).toBe("invalid_input");
    expect(await codeOf(move(a, mp, "in", 1.5))).toBe("invalid_input");
  });

  it("returns not_found for unknown or other-tenant market products", async () => {
    const a = await makeOrg("A");
    const b = await makeOrg("B");
    const theirs = await seedMarketProduct(b);
    expect(await codeOf(move(a, theirs.marketProductId, "in", 1))).toBe("not_found");
    expect(await codeOf(move(a, "00000000-0000-4000-8000-000000000000", "in", 1))).toBe("not_found");
  });

  it("keeps an exact balance under concurrent sales", async () => {
    const a = await makeOrg("A");
    const { marketProductId: mp } = await seedMarketProduct(a);
    await move(a, mp, "in", 50);
    await Promise.all(Array.from({ length: 20 }, () => move(a, mp, "sale", 1)));
    expect(await withOrg(a, (tx) => getBalance(tx, mp))).toBe(30);
  });

  it("lists active products at or below the minimum", async () => {
    const a = await makeOrg("A");
    const { marketId, productId, marketProductId: mp } = await seedMarketProduct(a);
    await withOrg(a, (tx) => setMarketProduct(tx, a, { marketId, productId, priceCents: 500, minStock: 5
        }));
    await move(a, mp, "in", 5);
    expect((await withOrg(a, (tx) => listLowStock(tx, marketId))).map((r) => r.id)).toEqual([mp]);
    await move(a, mp, "in", 1);
    expect(await withOrg(a, (tx) => listLowStock(tx, marketId))).toEqual([]);
    await move(a, mp, "loss", 6);
    await withOrg(a, (tx) => setMarketProduct(tx, a, { marketId, productId, priceCents: 500, minStock:
      5, active: false }));
    expect(await withOrg(a, (tx) => listLowStock(tx, marketId))).toEqual([]);
  });
});
```

[ ] Passo 3: Rodar e ver falhar

Executar: `npm test -- tests/domain/market-products.test.ts tests/domain/stock.test.ts`

Esperado: FAIL (módulos não existem).

[ ] Passo 4: Implementar `src/domain/market-products.ts`

```
import { z } from "zod";
import type { Tx } from "@/db/tenant";
import { AppError, isForeignKeyViolation } from "@/lib/errors";
import { parse } from "@/lib/validate";

const Input = z.object({
  marketId: z.string().uuid(),
  productId: z.string().uuid(),
  priceCents: z.number().int().min(1),
  minStock: z.number().int().min(0).default(0),
  active: z.boolean().default(true),
});

export type MarketProductRow = {
  id: string;
  productId: string;
  name: string;
  priceCents: number;
  minStock: number;
  active: boolean;
  balance: number;
};

export async function setMarketProduct(tx: Tx, orgId: string, raw: unknown): Promise<{ id: string }> {
  const i = parse(Input, raw);
  try {
    const [row] = await tx`
      insert into market_products (org_id, market_id, product_id, price_cents, min_stock, active)
      values (${orgId}, ${i.marketId}, ${i.productId}, ${i.priceCents}, ${i.minStock}, ${i.active})
      on conflict (market_id, product_id) do update
        set price_cents = excluded.price_cents, min_stock = excluded.min_stock, active = excluded.active
      returning id`;
    return { id: row.id as string };
  } catch (e) {
    if (isForeignKeyViolation(e)) throw new AppError("not_found");
    throw e;
  }
}

export async function listMarketProducts(tx: Tx, marketId: string): Promise<MarketProductRow[]> {
  const rows = await tx`
    select mp.id, mp.product_id, p.name, mp.price_cents, mp.min_stock, mp.active,
      coalesce((select sum(s.quantity) from stock_movements s where s.market_product_id = mp.id),
          0)::int as balance
    from market_products mp join products p on p.id = mp.product_id
    where mp.market_id = ${marketId} order by p.name`;
  return rows.map((r) => ({
    id: r.id as string,
    productId: r.product_id as string,
    name: r.name as string,
    priceCents: r.price_cents as number,
    minStock: r.min_stock as number,
    active: r.active as boolean,
    balance: r.balance as number,
  }));
}
```

[ ] Passo 5: Implementar `src/domain/stock.ts`

```
import { z } from "zod";
import type { Tx } from "@/db/tenant";
import { AppError, isForeignKeyViolation } from "@/lib/errors";
import { parse } from "@/lib/validate";
import { type MarketProductRow, listMarketProducts } from "./market-products";

const Movement = z
  .object({
    marketProductId: z.string().uuid(),
    type: z.enum(["in", "sale", "loss", "adjustment"]),
    quantity: z.number().int().refine((n) => n !== 0, "quantity must not be zero"),
    note: z.string().trim().max(200).optional(),
  })
  .superRefine((m, ctx) => {
    if (m.type !== "adjustment" && m.quantity < 0) {
      ctx.addIssue({ code: "custom", path: ["quantity"], message: "quantity must be positive" });
    }
  });

/**
 * in/sale/loss take a positive quantity (sign comes from the type);
 * adjustment takes a signed, non-zero quantity. Balance may go negative on purpose:
 * the ledger records what happened, adjustments fix it.
 */
export async function recordMovement(
  tx: Tx,
  orgId: string,
  userId: string | null,
  raw: unknown,
): Promise<{ id: string; delta: number }> {
  const m = parse(Movement, raw);
  const delta = m.type === "sale" || m.type === "loss" ? -m.quantity : m.quantity;
  try {
    const [row] = await tx`
      insert into stock_movements (org_id, market_product_id, type, quantity, note, created_by)
      values (${orgId}, ${m.marketProductId}, ${m.type}, ${delta}, ${m.note ?? null}, ${userId})
      returning id`;
    return { id: row.id as string, delta };
  } catch (e) {
    if (isForeignKeyViolation(e)) throw new AppError("not_found");
    throw e;
  }
}

export async function getBalance(tx: Tx, marketProductId: string): Promise<number> {
  const [r] = await tx`
    select coalesce(sum(quantity), 0)::int as balance
    from stock_movements where market_product_id = ${marketProductId}`;
  return r.balance as number;
}

export async function listLowStock(tx: Tx, marketId: string): Promise<MarketProductRow[]> {
  return (await listMarketProducts(tx, marketId)).filter((r) => r.active && r.balance <= r.minStock);
}
```

[ ] Passo 6: Rodar e ver passar

Executar: `npm test`

Esperado: PASS (toda a suíte).

[ ] Passo 7: Commit

```
git add -A
git commit -m "feat: add per-market pricing and stock ledger domain"
```

TAREFA 8

Interface web: entrar, cadastrar e navegação da organização

Arquivos:

- Criar: `src/app/layout.tsx`, `src/app/page.tsx`, `src/app/action-utils.ts`
- Criar: `src/app/signup/page.tsx`, `src/app/signup/actions.ts`
- Criar: `src/app/login/page.tsx`, `src/app/login/actions.ts`
- Criar: `src/app/o/[orgId]/layout.tsx`, `src/app/o/[orgId]/page.tsx`, `src/app/o/[orgId]/actions.ts`

Interfaces:

- Consome: `signUp`, `login`, `listMemberships`, `deleteSession`, `SESSION_COOKIE`, `requireUser`, `requireOrg`, `listMarkets`, `listLowStock`, `withOrg`, `message`, `AppError`, `formToObject`.
- Produz: `redirectOnAppError(e: unknown, path: string): never` (`@/app/action-utils`) usado por todas as actions; `setSessionCookie(token, expiresAt)` (`@/app/action-utils`); `logoutAction()` (`@/app/o/[orgId]/actions`).

Páginas são exercitadas no roteiro manual da Tarefa 10 (sem teste automatizado de UI nesta fase). Estilo visual é mínimo de propósito.

[ ] Passo 1: Criar `src/app/action-utils.ts`

```
import { cookies } from "next/headers";
import { redirect } from "next/navigation";
import { SESSION_COOKIE } from "@/auth/guard";
import { AppError } from "@/lib/errors";

/** Turns a domain AppError into a redirect with ?error=code; rethrows everything else
 *  (including Next's own redirect signals). */
export function redirectOnAppError(e: unknown, path: string): never {
  if (e instanceof AppError) redirect(`${path}?error=${e.code}`);
  throw e;
}

export async function setSessionCookie(token: string, expiresAt: Date): Promise<void> {
  (await cookies()).set(SESSION_COOKIE, token, {
    httpOnly: true,
    sameSite: "lax",
    secure: process.env.NODE_ENV === "production",
    expires: expiresAt,
    path: "/",
  });
}
```

[ ] Passo 2: Criar `src/app/layout.tsx` e `src/app/page.tsx`

`src/app/layout.tsx`:

```
export const metadata = { title: "Market77" };

export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (
    <html lang="pt-BR">
      <body>{children}</body>
    </html>
  );
}
```

`src/app/page.tsx`:

```
import { cookies } from "next/headers";
import { redirect } from "next/navigation";
import { SESSION_COOKIE } from "@/auth/guard";
import { listMemberships } from "@/auth/service";
import { getSessionUser } from "@/auth/sessions";

export default async function Home() {
  const token = (await cookies()).get(SESSION_COOKIE)?.value;
  const user = token ? await getSessionUser(token) : null;
  if (!user) redirect("/login");
  const [first] = await listMemberships(user.userId);
  redirect(first ? `/o/${first.orgId}` : "/login");
}
```

[ ] Passo 3: Criar cadastro (`src/app/signup/*`)

`src/app/signup/actions.ts`:

```
"use server";
import { redirect } from "next/navigation";
import { redirectOnAppError, setSessionCookie } from "@/app/action-utils";
import { signUp } from "@/auth/service";
import { formToObject } from "@/lib/validate";

export async function signUpAction(fd: FormData) {
  let orgId: string;
  try {
    const r = await signUp(formToObject(fd));
    await setSessionCookie(r.token, r.expiresAt);
    orgId = r.orgId;
  } catch (e) {
    redirectOnAppError(e, "/signup");
  }
  redirect(`/o/${orgId}`);
}
```

`src/app/signup/page.tsx`:

```
import { message } from "@/lib/messages";
import { signUpAction } from "./actions";

export default async function SignUpPage({ searchParams }: { searchParams: Promise<{ error?: string }>
    }) {
  const { error } = await searchParams;
  return (
    <main>
      <h1>Criar conta</h1>
      {error && <p role="alert">{message(error)}</p>}
      <form action={signUpAction}>
        <label>Nome da empresa <input name="orgName" required minLength={2} /></label>
        <label>E-mail <input name="email" type="email" required /></label>
        <label>Senha (mínimo 8 caracteres) <input name="password" type="password" required minLength={8}
            /></label>
        <button type="submit">Criar conta</button>
      </form>
      <a href="/login">Já tenho conta</a>
    </main>
  );
}
```

[ ] Passo 4: Criar login (`src/app/login/*`)

`src/app/login/actions.ts`:

```
"use server";
import { redirect } from "next/navigation";
import { redirectOnAppError, setSessionCookie } from "@/app/action-utils";
import { listMemberships, login } from "@/auth/service";
import { formToObject } from "@/lib/validate";

export async function loginAction(fd: FormData) {
  let target: string;
  try {
    const r = await login(formToObject(fd));
    await setSessionCookie(r.token, r.expiresAt);
    const [first] = await listMemberships(r.userId);
    target = first ? `/o/${first.orgId}` : "/login?error=forbidden";
  } catch (e) {
    redirectOnAppError(e, "/login");
  }
  redirect(target);
}
```

`src/app/login/page.tsx`:

```
import { message } from "@/lib/messages";
import { loginAction } from "./actions";

export default async function LoginPage({ searchParams }: { searchParams: Promise<{ error?: string }> })
    {
  const { error } = await searchParams;
  return (
    <main>
      <h1>Entrar</h1>
      {error && <p role="alert">{message(error)}</p>}
      <form action={loginAction}>
        <label>E-mail <input name="email" type="email" required /></label>
        <label>Senha <input name="password" type="password" required /></label>
        <button type="submit">Entrar</button>
      </form>
      <a href="/signup">Criar conta</a>
    </main>
  );
}
```

[ ] Passo 5: Criar a navegação da organização

`src/app/o/[orgId]/actions.ts`:

```
"use server";
import { cookies } from "next/headers";
import { redirect } from "next/navigation";
import { SESSION_COOKIE } from "@/auth/guard";
import { deleteSession } from "@/auth/sessions";

export async function logoutAction() {
  const jar = await cookies();
  const token = jar.get(SESSION_COOKIE)?.value;
  if (token) await deleteSession(token);
  jar.delete(SESSION_COOKIE);
  redirect("/login");
}
```

`src/app/o/[orgId]/layout.tsx`:

```
import { requireOrg } from "@/auth/guard";
import { can } from "@/auth/permissions";
import { logoutAction } from "./actions";

export default async function OrgLayout({
  children,
  params,
}: {
  children: React.ReactNode;
  params: Promise<{ orgId: string }>;
}) {
  const { orgId } = await params;
  const { role } = await requireOrg(orgId);
  const base = `/o/${orgId}`;
  return (
    <div>
      <nav>
        <a href={base}>Painel</a> <a href={`${base}/markets`}>Mercados</a>{" "}
        <a href={`${base}/products`}>Produtos</a>{" "}
        {can(role, "member:write") && <a href={`${base}/team`}>Equipe</a>}
        <form action={logoutAction} style={{ display: "inline" }}>
          <button type="submit">Sair</button>
        </form>
      </nav>
      {children}
    </div>
  );
}
```

`src/app/o/[orgId]/page.tsx` (painel: mercados e alertas de estoque baixo):

```
import { requireOrg } from "@/auth/guard";
import { withOrg } from "@/db/tenant";
import { listMarkets } from "@/domain/markets";
import { listLowStock } from "@/domain/stock";

export default async function Dashboard({ params }: { params: Promise<{ orgId: string }> }) {
  const { orgId } = await params;
  await requireOrg(orgId, "stock:read");
  const markets = await withOrg(orgId, async (tx) =>
    Promise.all((await listMarkets(tx)).map(async (m) => ({ ...m, low: await listLowStock(tx, m.id)
        }))),
  );
  return (
    <main>
      <h1>Painel</h1>
      {markets.length === 0 && <p>Nenhum mercado cadastrado ainda.</p>}
      <ul>
        {markets.map((m) => (
          <li key={m.id}>
            <a href={`/o/${orgId}/markets/${m.id}`}>{m.name}</a>
            {m.low.length > 0 && <strong> ({m.low.length} produto(s) com estoque baixo)</strong>}
          </li>
        ))}
      </ul>
    </main>
  );
}
```

[ ] Passo 6: Typecheck e commit

Executar: `npm run typecheck`

Esperado: sem erros.

```
git add -A
git commit -m "feat: add auth pages and organization shell"
```

TAREFA 9

Interface web: mercados, produtos, estoque e equipe

Arquivos:

- Criar: `src/app/o/[orgId]/markets/page.tsx`, `src/app/o/[orgId]/markets/actions.ts`
- Criar: `src/app/o/[orgId]/markets/[marketId]/page.tsx`, `src/app/o/[orgId]/markets/[marketId]/actions.ts`
- Criar: `src/app/o/[orgId]/products/page.tsx`, `src/app/o/[orgId]/products/actions.ts`
- Criar: `src/app/o/[orgId]/team/page.tsx`, `src/app/o/[orgId]/team/actions.ts`

Interfaces:

- Consome: tudo das Tarefas 2 a 8 (`requireOrg`, `withOrg`, `createMarket`, `listMarkets`, `createProduct`, `listProducts`, `setMarketProduct`, `listMarketProducts`, `recordMovement`, `addMember`, `parseBRL`, `formatBRL`, `formToObject`, `redirectOnAppError`, `message`).
- Regra: cada action chama `requireOrg(orgId, <ação>)` antes de qualquer escrita (matriz da Tarefa 4).

[ ] Passo 1: Mercados

`src/app/o/[orgId]/markets/actions.ts`:

```
"use server";
import { redirect } from "next/navigation";
import { redirectOnAppError } from "@/app/action-utils";
import { requireOrg } from "@/auth/guard";
import { withOrg } from "@/db/tenant";
import { createMarket } from "@/domain/markets";
import { formToObject } from "@/lib/validate";
export async function createMarketAction(orgId: string, fd: FormData) {
  const path = `/o/${orgId}/markets`;
  try {
    await requireOrg(orgId, "market:write");
    await withOrg(orgId, (tx) => createMarket(tx, orgId, formToObject(fd)));
  } catch (e) {
    redirectOnAppError(e, path);
  }
  redirect(path);
}
```

`src/app/o/[orgId]/markets/page.tsx`:

```
import { requireOrg } from "@/auth/guard";
import { can } from "@/auth/permissions";
import { withOrg } from "@/db/tenant";
import { listMarkets } from "@/domain/markets";
import { message } from "@/lib/messages";
import { createMarketAction } from "./actions";

export default async function MarketsPage({
  params,
  searchParams,
}: {
  params: Promise<{ orgId: string }>;
  searchParams: Promise<{ error?: string }>;
}) {
  const { orgId } = await params;
  const { error } = await searchParams;
```

```
  const { role } = await requireOrg(orgId, "catalog:read");
  const markets = await withOrg(orgId, (tx) => listMarkets(tx));
  return (
    <main>
      <h1>Mercados</h1>
      {error && <p role="alert">{message(error)}</p>}
      <ul>
        {markets.map((m) => (
          <li key={m.id}><a href={`/o/${orgId}/markets/${m.id}`}>{m.name}</a> {m.address}</li>
        ))}
      </ul>
      {can(role, "market:write") && (
        <form action={createMarketAction.bind(null, orgId)}>
          <h2>Novo mercado</h2>
          <label>Nome <input name="name" required minLength={2} /></label>
          <label>Endereço <input name="address" /></label>
          <label>CNPJ <input name="cnpj" inputMode="numeric" /></label>
          <button type="submit">Cadastrar</button>
        </form>
      )}
    </main>
  );
}
```

[ ] Passo 2: Produtos

`src/app/o/[orgId]/products/actions.ts`:

```
"use server";
import { redirect } from "next/navigation";
import { redirectOnAppError } from "@/app/action-utils";
import { requireOrg } from "@/auth/guard";
import { withOrg } from "@/db/tenant";
import { createProduct } from "@/domain/products";
import { formToObject } from "@/lib/validate";
export async function createProductAction(orgId: string, fd: FormData) {
  const path = `/o/${orgId}/products`;
  try {
    await requireOrg(orgId, "product:write");
    const raw = formToObject(fd);
    const barcodes = (raw.barcodes ?? "").split(/[\s,;]+/).filter(Boolean);
    await withOrg(orgId, (tx) => createProduct(tx, orgId, { ...raw, barcodes }));
  } catch (e) {
    redirectOnAppError(e, path);
  }
  redirect(path);
}
```

`src/app/o/[orgId]/products/page.tsx`:

```
import { requireOrg } from "@/auth/guard";
import { can } from "@/auth/permissions";
import { withOrg } from "@/db/tenant";
import { listProducts } from "@/domain/products";
import { message } from "@/lib/messages";
import { createProductAction } from "./actions";

export default async function ProductsPage({
  params,
  searchParams,
}: {
  params: Promise<{ orgId: string }>;
  searchParams: Promise<{ error?: string }>;
}) {
  const { orgId } = await params;
  const { error } = await searchParams;
```

```
  const { role } = await requireOrg(orgId, "catalog:read");
  const products = await withOrg(orgId, (tx) => listProducts(tx));
  return (
    <main>
      <h1>Produtos</h1>
      {error && <p role="alert">{message(error)}</p>}
      <table>
        <thead><tr><th>Nome</th><th>Categoria</th><th>Códigos de barras</th></tr></thead>
        <tbody>
          {products.map((p) => (
            <tr key={p.id}><td>{p.name}</td><td>{p.category}</td><td>{p.barcodes.join(", ")}</td></tr>
          ))}
        </tbody>
      </table>
      {can(role, "product:write") && (
        <form action={createProductAction.bind(null, orgId)}>
          <h2>Novo produto</h2>
          <label>Nome <input name="name" required minLength={2} /></label>
          <label>Categoria <input name="category" /></label>
          <label>Códigos de barras (separe por vírgula) <input name="barcodes" /></label>
          <button type="submit">Cadastrar</button>
        </form>
      )}
    </main>
  );
}
```

[ ] Passo 3: Detalhe do mercado (preço, mínimo e movimentação de estoque)

`src/app/o/[orgId]/markets/[marketId]/actions.ts`:

```
"use server";
import { redirect } from "next/navigation";
import { redirectOnAppError } from "@/app/action-utils";
import { requireOrg } from "@/auth/guard";
import { withOrg } from "@/db/tenant";
import { setMarketProduct } from "@/domain/market-products";
import { recordMovement } from "@/domain/stock";
import { AppError } from "@/lib/errors";
import { parseBRL } from "@/lib/money";
import { formToObject } from "@/lib/validate";

export async function setMarketProductAction(orgId: string, marketId: string, fd: FormData) {
  const path = `/o/${orgId}/markets/${marketId}`;
  try {
    await requireOrg(orgId, "price:write");
    const raw = formToObject(fd);
    const priceCents = parseBRL(raw.price ?? "");
    if (priceCents === null) throw new AppError("invalid_input", "invalid price");
    await withOrg(orgId, (tx) =>
      setMarketProduct(tx, orgId, {
        marketId,
        productId: raw.productId,
        priceCents,
        minStock: Number(raw.minStock ?? 0),
        active: raw.active === "on",
      }),
    );
  } catch (e) {
    redirectOnAppError(e, path);
  }
  redirect(path);
}
export async function moveStockAction(orgId: string, marketId: string, marketProductId: string, fd:
    FormData) {
  const path = `/o/${orgId}/markets/${marketId}`;
  try {
    const { userId } = await requireOrg(orgId, "stock:move");
    const raw = formToObject(fd);
    await withOrg(orgId, (tx) =>
```

```
      recordMovement(tx, orgId, userId, {
        marketProductId,
        type: raw.type,
        quantity: Number(raw.quantity),
        note: raw.note,
      }),
    );
  } catch (e) {
    redirectOnAppError(e, path);
  }
  redirect(path);
}
```

src/app/o/[orgId]/markets/[marketId]/page.tsx:

```
import { requireOrg } from "@/auth/guard";
import { can } from "@/auth/permissions";
import { withOrg } from "@/db/tenant";
import { listMarketProducts } from "@/domain/market-products";
import { listProducts } from "@/domain/products";
import { message } from "@/lib/messages";
import { formatBRL } from "@/lib/money";
import { moveStockAction, setMarketProductAction } from "./actions";
export default async function MarketPage({
  params,
  searchParams,
}: {
  params: Promise<{ orgId: string; marketId: string }>;
  searchParams: Promise<{ error?: string }>;
}) {
  const { orgId, marketId } = await params;
  const { error } = await searchParams;
  const { role } = await requireOrg(orgId, "stock:read");
  const { rows, products } = await withOrg(orgId, async (tx) => ({
    rows: await listMarketProducts(tx, marketId),
    products: await listProducts(tx),
  }));
  return (
    <main>
      <h1>Estoque do mercado</h1>
      {error && <p role="alert">{message(error)}</p>}
      <table>
        <thead>

              <tr><th>Produto</th><th>Preço</th><th>Mínimo</th><th>Saldo</th><th>Ativo</th><th>Movimenta
              r</th></tr>
        </thead>
        <tbody>
          {rows.map((r) => (
            <tr key={r.id}>
              <td>{r.name}</td>
              <td>{formatBRL(r.priceCents)}</td>
              <td>{r.minStock}</td>
              <td>{r.balance <= r.minStock ? <strong>{r.balance} (baixo)</strong> : r.balance}</td>
              <td>{r.active ? "sim" : "não"}</td>
              <td>
                {can(role, "stock:move") && (
                  <form action={moveStockAction.bind(null, orgId, marketId, r.id)}>
                    <select name="type" defaultValue="in">
                      <option value="in">Entrada</option>
                      <option value="sale">Venda</option>
                      <option value="loss">Perda</option>
                      <option value="adjustment">Ajuste (use sinal)</option>
                    </select>
                    <input name="quantity" type="number" step="1" required />
                    <input name="note" placeholder="Observação" />
                    <button type="submit">Registrar</button>
                  </form>
                )}
              </td>
```

```
            </tr>
          ))}
        </tbody>
      </table>
      {can(role, "price:write") && (
        <form action={setMarketProductAction.bind(null, orgId, marketId)}>
          <h2>Adicionar ou atualizar produto neste mercado</h2>
          <label>
            Produto{" "}
            <select name="productId" required>
              {products.map((p) => <option key={p.id} value={p.id}>{p.name}</option>)}
            </select>
          </label>
          <label>Preço (R$) <input name="price" required inputMode="decimal" placeholder="5,90"
              /></label>
          <label>Estoque mínimo <input name="minStock" type="number" min={0} defaultValue={0} /></label>
          <label>Ativo <input name="active" type="checkbox" defaultChecked /></label>
          <button type="submit">Salvar</button>
        </form>
      )}
    </main>
  );
}
```

[ ] Passo 4: Equipe

src/app/o/[orgId]/team/actions.ts:

```
"use server";
import { redirect } from "next/navigation";
import { redirectOnAppError } from "@/app/action-utils";
import { requireOrg } from "@/auth/guard";
import { addMember } from "@/auth/service";
import { formToObject } from "@/lib/validate";

export async function addMemberAction(orgId: string, fd: FormData) {
  const path = `/o/${orgId}/team`;
  try {
    await requireOrg(orgId, "member:write");
    await addMember(orgId, formToObject(fd));
  } catch (e) {
    redirectOnAppError(e, path);
  }
  redirect(`${path}?ok=1`);
}
```

src/app/o/[orgId]/team/page.tsx:

```
import { requireOrg } from "@/auth/guard";
import { message } from "@/lib/messages";
import { addMemberAction } from "./actions";

export default async function TeamPage({
  params,
  searchParams,
}: {
  params: Promise<{ orgId: string }>;
  searchParams: Promise<{ error?: string; ok?: string }>;
}) {
  const { orgId } = await params;
  const { error, ok } = await searchParams;
  await requireOrg(orgId, "member:write");
  return (
    <main>
```

```
      <h1>Equipe</h1>
      {error && <p role="alert">{message(error)}</p>}
      {ok && <p>Membro adicionado.</p>}
      <form action={addMemberAction.bind(null, orgId)}>
        <label>E-mail <input name="email" type="email" required /></label>
        <label>Senha inicial (só para usuário novo) <input name="password" type="password" minLength={8}
             /></label>
        <label>
          Papel{" "}
          <select name="role">
            <option value="manager">Gerente</option>
            <option value="restocker">Repositor</option>
          </select>
        </label>
        <button type="submit">Adicionar</button>
      </form>
    </main>
  );
}
```

[ ] Passo 5: Typecheck e commit

Executar: `npm run typecheck`

Esperado: sem erros.

```
git add -A
git commit -m "feat: add markets, products, stock and team pages"
```

TAREFA 10

Verificação final e README

Arquivos:

- Criar: `README.md`

[ ] Passo 1: Escrever `README.md`

```
# Market77

Plataforma SaaS para mercados autônomos. Fase 1: base multi-tenant, catálogo e estoque.

## Rodar localmente

1. `docker compose up -d`
2. `cp .env.example .env`
3. `npm install`
4. `npm run db:migrate`
5. `npm run dev` e abrir http://localhost:3000

## Testes

`docker compose up -d && npm test` (usa o banco `market77_test`).
## Produção

Troque a senha do papel `market77_app` (`ALTER ROLE market77_app PASSWORD '...'`) e use essa senha em
    `DATABASE_APP_URL`.
```

[ ] Passo 2: Rodar a suíte completa e o typecheck

Executar: `npm test && npm run typecheck`

Esperado: todos os testes PASS e typecheck sem erros.

[ ] Passo 3: Roteiro manual (com `npm run db:migrate && npm run dev`)

1. Em `/signup`, criar conta (empresa "Mercados Ana", e-mail e senha de 8+ caracteres). Esperado: cair no Painel vazio.
2. Em Mercados, cadastrar "Loja Bloco 1" e abrir o mercado.
3. Em Produtos, cadastrar "Água 500ml" com código `7891000100103`. Tentar cadastrar outro produto com o mesmo código: deve mostrar "Este código de barras já está em uso."
4. No mercado, adicionar a Água com preço `5,90` e mínimo `5`. Registrar entrada de 10 e venda de 6: saldo 4 e aviso "baixo"; o Painel deve mostrar "1 produto(s) com estoque baixo".
5. Em Equipe, adicionar um repositor com senha inicial. Sair, entrar como o repositor: deve ver estoque e movimentar, mas não deve ver o formulário de preço nem o link Equipe.
6. Criar uma segunda conta em outra janela anônima e abrir a URL `/o/<orgId da primeira>`: deve receber erro de permissão, sem vazar dados.
7. Reiniciar o servidor (`Ctrl+C`, `npm run dev`) e confirmar que a sessão continua válida.

[ ] Passo 4: Commit

```
git add -A
git commit -m "docs: add README with local setup"
```

Auto-revisão

Cobertura da spec (fase 1): organizations, users, memberships e papéis (Tarefas 2, 4, 5); markets com dados fiscais básicos (Tarefas 2 e 6); products e `product_barcodes` (Tarefas 2 e 6); `market_products` com preço por mercado (Tarefa 7); `stock_movements` como livro-razão com saldo derivado (Tarefas 2 e 7); RLS em todas as tabelas de negócio (Tarefa 2); painel do dono (Tarefas 8 e 9). Fica para fases seguintes, por decisão da spec: `terminals`, `terminal_commands`, `stock_lots` (lote e validade), `sales`, `payments`, `fiscal_documents`, `provider_credentials`, `access_events`.

Pontos de atenção para quem executa:

- Tipos e nomes são consistentes entre tarefas: `Tx`, `withOrg`, `Role`, `Action`, `can`, `MarketProductRow`, `recordMovement(tx, orgId, userId, raw)`.
- O código de barras aceita `[0-9A-Za-z-]{4,32}`; ajuste se algum fornecedor usar outro formato.
- Falha de permissão hoje mostra a página de erro padrão do Next; refinar na fase de acabamento da interface.
- `next build` exige as variáveis de ambiente do banco presentes (o client valida na importação).